Base de Datos I

Funciones y Triggers

Centro de e-Learning
UTN.BA

Funciones



- Las funciones permiten encapsular lógica reutilizable de forma tal que, a partir de su invocación, se pueda obtener un resultado.
- Las funciones pueden recibir parámetros y devolver un único valor como resultado.
- Los valores pueden ser de tipos de datos nativos o pueden ser de tipo registro; este último permite que una función devuelva, tal como mencionamos en otras unidades, una tabla.
- Las funciones permiten también la reutilización de la base de datos ya que otras aplicaciones pueden interactuar directamente con ellas en lugar de hacerlo a través de código u otras consultas, garantizando de esta forma un comportamiento homogéneo.
- PostgreSQL permite desarrollar funciones utilizando distintos lenguajes, como SQL, PL/pgSQL, C o Python.
- PL/pgSQL es el lenguaje procedural nativo de PostgreSQL y permite incorporar lógica de programación dentro de funciones y procedimientos.

Centro de e-Learning UTN.BA

Sintaxis

```
CREATE OR REPLACE FUNCTION <nombre>([[<mode>] <arg1> <type>, ...,[<mode>]
<argN> <type>]) RETURNS <ret_type> AS $$
DECLARE <var_name> <var_type>
BEGIN
...;
END;
$$ LANGUAGE <language>;
```

- **mode**: IN|OUT|INOUT
- **type**: tipo de cada argumento. Si el tipo está basado en alguna columna, se puede utilizar la siguiente expresión: <table_name>.<column_name>%TYPE
- **ret_type**: tipo de retorno de la función. Puede ser un tipo básico o TABLE, en este caso hay que especificar las columnas y sus tipos.
- **language**: puede ser sql; plpgsql; plpythonu

Centro de e-Learning UTN.BA

Tipos de funciones

- Los tipos de funciones están determinados principalmente por el lenguaje que se utilice para programarlas.
- En el caso de ***SQL*** sólo podrán efectuarse operaciones especificadas en dicho lenguaje. Si requerimos especificar una lógica más compleja, podemos utilizar ***plpgsql*** que nos habilita el uso de estructuras de decisión, bucles, etc.
- También existe el tipo ***plpythonu*** que nos permite escribir las funciones en python, con todo su potencial.

```
-- Ejemplo de función que permite actualizar un registro de una tabla
CREATE FUNCTION tf1 (accountno integer, debit numeric) RETURNS integer AS $$
  UPDATE bank
    SET balance = balance - debit
    WHERE accountno = tf1.accountno;
  SELECT 1;
$$ LANGUAGE SQL;


-- Ejemplo de función cuyo tipo de retorno es una tabla con 2 campos
CREATE FUNCTION dup(int) RETURNS TABLE(f1 int, f2 text) AS $$
  SELECT $1, CAST($1 AS text) || ' is text'
$$ LANGUAGE SQL;


SELECT * FROM dup(42);

-- Genera la siguiente salida
 f1 |     f2
----+------------
 42 | 42 is text
(1 row)
```


Centro de e-Learning UTN.BA

Utilizando lenguaje plpgsql:

```
-- Ejemplo de función que valida un par de credenciales
CREATE OR REPLACE FUNCTION check_password(uname TEXT, pass TEXT) RETURNS BOOLEAN AS
$$
DECLARE passed BOOLEAN;
BEGIN
  passed := false;
  SELECT (password = $2) INTO passed
  FROM cliente
  WHERE name = $1;

  -- en caso de que el query no retorne filas, la variable passed contiene NULL.
  IF passed IS NULL THEN
      passed := false;
  END IF;

  RETURN passed;
END;
$$ LANGUAGE plpgsql;
```

```
-- Ejemplo de función que suma 2 enteros si y sólo si ambos son positivos
CREATE FUNCTION sum_positive(n1 integer, n2 integer) RETURNS integer AS $$
DECLARE sum integer;
BEGIN
  IF(n1 > 0 AND n2 > 0) THEN
    RETURN n1+n2;
  ELSE
    RAISE NOTICE 'Error!!';
  END IF;
END;
$$ LANGUAGE plpgsql;
```

```
-- Ejemplo de función que devuelve el registro con el id mayor.
CREATE OR REPLACE FUNCTION last_record() RETURNS provincias AS $$
DECLARE  t2_row provincias%ROWTYPE;
BEGIN
       select * into t2_row
       from provincias order by prv_id desc limit 1;

       return t2_row;
END;

$$ LANGUAGE plpgsql;
```


Centro de e-Learning UTN.BA

Excepciones

```
RAISE [nivel] [mensaje]
```

- **nivel**: permite indicar el nivel de error. Puede ser uno de DEBUG, LOG, INFO, NOTICE, WARNING, o EXCEPTION. El nivel EXCEPTION es la opción por defecto y es el único de todos los niveles que permite cortar la ejecución de la función.
- **mensaje**: permite generar un mensaje para describir el error. Cabe destacar que en el archivo *postgresql.conf* podemos editar el nivel de los mensajes que deseamos registrar. Se puede configurar las variables *log_min_messages* y *client_min_messages* para configurar el nivel de los mensajes a reportar en el archivo de log y al cliente respectivamente.

Centro de e-Learning UTN.BA

Ejemplo de excepción

```
-- Ejemplo de función que valida un par de credenciales
CREATE OR REPLACE FUNCTION check_password(uname TEXT, pass TEXT) RETURNS BOOLEAN AS
$$
DECLARE passed BOOLEAN;
BEGIN
  passed := false;
  SELECT (password = $2) INTO passed
  FROM cliente
  WHERE name = $1;

  -- en caso de que el query no retorne filas, la variable passed contiene NULL.
  IF passed IS NULL THEN
```

```
    RAISE LOG 'El usuario % no existe', uname;
      passed := false;
  END IF;

  RETURN passed;
END;
$$ LANGUAGE plpgsql;


-- Al ejecutar la función
select check_password('juan', 'clave_1234');

-- Obtenemos la salida
 check_password
----------------
 f
(1 row)


-- Y si vemos el archivo de log, veremos el siguiente mensaje
2021-10-28 03:45:13.093 UTC [49] LOG:  El usuario juan no existe
2021-10-28 03:45:13.093 UTC [49] CONTEXT:  PL/pgSQL function
check_password(text,text) line 10 at RAISE
2021-10-28 03:45:13.093 UTC [49] STATEMENT:  select check_password('juan',
'clave_1234');
```


Centro de e-Learning UTN.BA

Triggers



- Es una indicación que podemos proporcionar al motor de base de datos para que ejecute una función ante determinado evento.
- La función puede ser de cualquier tipo (sql, plpsql, plpythonu, etc) y debe haber sido creada previamente a la definición del trigger.
- El tipo de retorno de la función debe ser trigger.

Centro de e-Learning UTN.BA

Tipos y Usos de triggers

When	Event	Row-level	Statement-level
Before	insert/update/delete	Tables	Tables + Views
	truncate	-	Tables
After	insert/update/delete	Tables	Tables + Views
	truncate	-	Tables
Instead Of	insert/update/delete	Views	-
	truncate	-	-

- Sobre las tablas los tipos de triggers soportados son:
- BEFORE: se ejecuta antes que el evento para el cual se definió
- AFTER : se ejecuta inmediatamente después que el evento para el cual se definió

Centro de e-Learning UTN.BA

Registros especiales

- El motor de base de datos inyecta a estas funciones, de forma automática, algunos datos (registros) que dependen del evento.
- **Registro NEW**
  - Este registro contiene la información de los datos a insertar o actualizar en la tabla en cuestión.
  - Es inyectado en las operaciones de insert o update sólo para triggers que se ejecutan a nivel de filas.
  - Es NULL en operaciones de delete o para triggers que se ejecutan por operación.
- **Registro OLD**
  - Este registro contiene la información de los datos eliminados en la tabla en cuestión.
  - Es inyectado en las operaciones de delete sólo para triggers que se ejecutan a nivel de filas.
  - Es NULL en operaciones insert o update o para triggers que se ejecutan por operación.

Centro de e-Learning UTN.BA

Creación de triggers

```
CREATE [ CONSTRAINT ] TRIGGER name { BEFORE | AFTER | INSTEAD OF } { event [ OR
... ] }
ON table
[ FROM referenced_table_name ]
[ NOT DEFERRABLE | [ DEFERRABLE ] { INITIALLY IMMEDIATE | INITIALLY
DEFERRED } ]
[ REFERENCING { { OLD | NEW } TABLE [ AS ] transition_relation_name } [ ... ] ]
[ FOR [ EACH ] { ROW | STATEMENT } ]
[ WHEN ( condition ) ]
EXECUTE PROCEDURE function_name ( arguments )
```

- PostgreSQL admite especificar una condición booleana en la definición del trigger sobre la cual basa su decisión para ejecutar el trigger o no. Esta condición se especifica mediante la palabra *WHEN*
- **Nota**: si se definen varios triggers sobre la misma tabla y para el mismo evento, los mismos se ejecutarán en orden alfabético.
- **Nota II**: puede existir recursividad en un trigger. Es responsabilidad del programador que no se creen loops infinitos.

Centro de e-Learning UTN.BA

Ejemplos de triggers

```
CREATE TRIGGER check_update
BEFORE UPDATE ON accounts
FOR EACH ROW
EXECUTE PROCEDURE check_account_update();
```

```
CREATE TRIGGER check_update
BEFORE UPDATE ON accounts
FOR EACH ROW
WHEN (OLD.balance IS DISTINCT FROM NEW.balance)
EXECUTE PROCEDURE check_account_update();
```

```
CREATE OR REPLACE FUNCTION before_insert_provincias() RETURNS trigger AS $$

BEGIN
       RAISE INFO 'ejecuta trigger';
       IF length(new.prv_nombre) < 3 THEN
          return null;
       END IF;
       return new;
END;

$$ LANGUAGE plpgsql;


CREATE TRIGGER before_insert_provincias
BEFORE INSERT ON provincias
FOR EACH ROW
EXECUTE PROCEDURE befor_insert_provincias();
```


Centro de e-Learning UTN.BA

¡Gracias!

Centro de e-Learning

UTN.BA